WAREHOUSE DELIVERY ROBOT

Search Algorithms: BFS and A*

COURSE	Intelligent Systems
ASSIGNMENT	Task 1 - Search Algorithms

METHODS

Breadth-First Search (BFS) and A* Search

IMPLEMENTATION

Python 3 | Three warehouse grid maps

Abstract

This report evaluates Breadth-First Search (BFS) and A* Search for routing a warehouse delivery robot on grid maps containing walls, normal floor cells, and rough floor cells. Both algorithms were evaluated on three supplied maps using path length, total path cost, expanded nodes, maximum frontier size, and execution time. The results demonstrate why minimizing the number of moves does not always minimize travel cost.

Project scope

The robot moves one cell at a time, only up, down, left, or right. Paths must remain within the grid and cannot pass through walls. Each valid move incurs the cost of the destination cell. The initial start cell is not charged.

1. Problem Formulation

State: a traversable grid cell represented by the coordinate pair (row, column), using zero-based indexing.

Initial state: the location of S, determined by scanning the selected grid.

Goal test: the current position equals the location of G.

Actions: Up (-1, 0), Down (+1, 0), Left (0, -1), Right (0, +1). Moves outside the grid or into # are invalid.

Transition: from (r, c) to the adjacent traversable cell (r + dr, c + dc).

Path cost: sum of costs of every cell entered, excluding S. Normal floor (.) and goal (G) cost 1; rough floor (~) costs 3; walls (#) are not traversable.

SYMBOL	DESCRIPTION	ENTRY COST
S	Start	Excluded initially
G	Goal	1
.	Normal floor	1
~	Rough floor	3
#	Wall	Not allowed

2. Search Algorithms

2.1 Breadth-First Search (BFS)

BFS is an uninformed search algorithm. It explores cells in increasing order of distance from the start using a First-In, First-Out (FIFO) queue. The visited set prevents duplicate discovery, and a parent dictionary reconstructs the path. BFS minimizes the number of moves on this grid but does not account for rough-floor costs while searching.

2.2 A* Search

A* is an informed search algorithm that uses a priority queue ordered by **f(n) = g(n) + h(n)**, where g(n) is the known cost from S to n and h(n) estimates the remaining cost to G. When a cheaper path to a cell is discovered, the best cost and parent are updated. Outdated heap entries are ignored.

2.3 Manhattan Distance Heuristic

h(n) = |row(n) - row(G)| + |col(n) - col(G)|. This heuristic matches four-direction movement and is admissible and consistent here because any move costs at least 1. As a result, A* can find a minimum-cost route under the implemented cost model.

3. Experimental Setup and Results

Both algorithms were run on the three maps supplied with the assignment. Neighbor exploration order was Up, Down, Left, Right. Path length counts moves; expanded nodes count valid queue/heap removals, including the goal. Maximum frontier size is the peak stored queue/heap entry count.

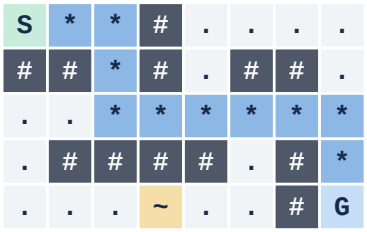
3.1 Final Comparison Table

Map	Algorithm	Length	Cost	Expanded	Max frontier	Time (ms)
MAP_A	BFS	11	15	28	5	0.0466
MAP_A	A*	11	15	21	5	0.0682
MAP_B	BFS	8	22	25	4	0.0357
MAP_B	A*	12	12	15	7	0.0437
MAP_C	BFS	N/A	N/A	13	3	0.0208
MAP_C	A*	N/A	N/A	13	3	0.0320

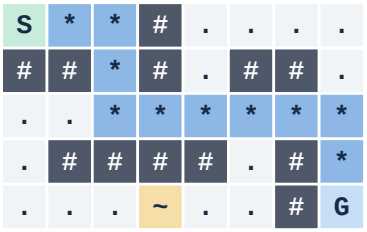
Execution times are taken from the user-provided terminal screenshot (one run). Millisecond-level measurements on these small maps are noisy and should not be treated as statistically conclusive.

3.2 Path Visualizations

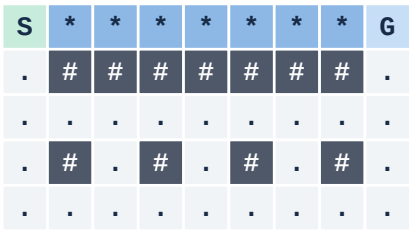
MAP_A - BFS | Moves: 11 | Cost: 15



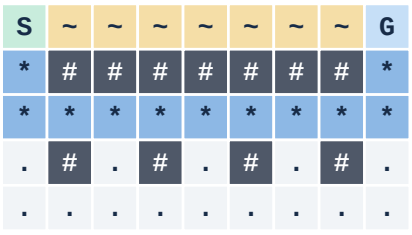
MAP_A - A* | Moves: 11 | Cost: 15



MAP_B - BFS | Moves: 8 | Cost: 22



MAP_B - A* | Moves: 12 | Cost: 12



*Colored cells marked * show intermediate steps; S and G remain visible. Blue cells represent the chosen path, gold cells rough floor, and dark cells walls. Diagrams are reconstructed from verified algorithm outputs.*

3.3 No-Solution Case: MAP_C

Neither BFS nor A* could reach G in MAP_C. Both algorithms exhausted the reachable search region and correctly returned "No path found". Each expanded 13 nodes and had a maximum frontier size of 3.

MAP_C - no path exists

S	.	.	#	.	.	.	.
.	#	.	#	.	#	#	.
.	.	.	#	.	.	#	.
.	#	#	#	.	#	#	.
.	.	.	.	#	.	.	G

3.4 Terminal Results Evidence

```
================================================================================
FINAL COMPARISON TABLE
================================================================================
Map       Algorithm   Length    Cost      Expanded    Frontier    Time(ms)
--------------------------------------------------------------------------------
MAP_A     BFS         11        15        28          5           0.0466
MAP_A     A*          11        15        21          5           0.0682
MAP_B     BFS         8         22        25          4           0.0357
MAP_B     A*          12        12        15          7           0.0437
MAP_C     BFS         N/A       N/A       13          3           0.0208
MAP_C     A*          N/A       N/A       13          3           0.0320
```

Figure: comparison table screenshot supplied from the user's local program execution.

4. Comparison and Discussion

MAP_A: both algorithms returned a path of 11 moves and cost 15. A* expanded 21 nodes versus 28 for BFS, while both reached a maximum frontier size of 5.

MAP_B: BFS chose the direct top-row route: 8 moves, cost 22. A* avoided the seven rough-floor cells by moving around the lower corridor: 12 moves, cost 12. A* reduced movement cost by 10 units, approximately 45.5%, while expanding 15 instead of 25 nodes.

MAP_C: no valid route exists from S to G. Both algorithms returned no solution after expanding 13 nodes.

The relative run-time ordering varies across the supplied single-run measurements, so the strongest conclusions concern route quality and search effort rather than speed.

5. Conclusion

The tests illustrate an important difference between shortest and cheapest paths. BFS guarantees the smallest number of moves on an unweighted grid, but those moves may traverse expensive terrain. With Manhattan distance and accumulated movement cost, A* finds optimal-cost routes under the assignment's movement rules. A* is therefore the more suitable choice for the warehouse delivery robot when normal and rough floors have different costs.

6. Implementation and Reproducibility

Programming language: Python 3. The implementation uses the standard-library modules collections.deque, heapq, itertools.count, and time.perf_counter; no pathfinding libraries are needed.

Source file: main.py (the student's working file). The implementation tested for this report is equivalent to the provided warehouse_search.py reference code.

Run command: `python main.py`

Maps: MAP_A, MAP_B, and MAP_C are defined as lists of strings near the top of the source file; run_tests() evaluates all three automatically. GRID may be changed for separate single-map experiments.

Outputs: path coordinates, path length, total cost, expanded nodes, maximum frontier size, run time, ASCII path visualization, and final comparison table.

7. Project Files and Deliverables

The project deliverables consist of the Python source code (main.py), a README with instructions for running the program, output screenshots for each test map, and this report. The implementation includes BFS, A* with the Manhattan distance heuristic, automated testing for MAP_A, MAP_B and MAP_C, path visualization, and the final comparison table.

Reference

Intelligent Systems - Task 1: Search Algorithms, Warehouse Delivery Robot,
Faculty of Computers and Data Science, Alexandria University.

GitHub Repository

https://github.com/27nsg6x892-afk/Warehouse-Delivery-Robot